

Odds exports — tennis player props

Generated September 19, 2026 from the nightly capture archive at `odds_snapshots/tennis_props_2026-09.csv`.

Two views of the same data. Both use LF line endings, a single header row, and quote any field containing a comma, so they load cleanly into Postgres COPY, pandas, Excel, or Power Query without fixing anything first.

File	Rows	Shape
<code>tennis_prop_odds_raw_2026-09.csv</code>	224	One row per price, exactly as captured. Over and Under are separate rows.
<code>tennis_prop_odds_pivoted_2026-09.csv</code>	118	One row per player/line/book, with Over and Under on the same row. Matches the shape of the Odds table.

Coverage

- **Captures:** 3 nights — 2026-09-18, 2026-09-19, 2026-09-20 (all at roughly 03:00 UTC).
- **Price timestamps:** 2026-09-17T20:07:54Z through 2026-09-20T02:56:04Z.
- **Leagues:** WTA only. No ATP or ATP Challenger prices have been returned on any night, although Challenger fixtures were found.
- **Books:** DraftKings (108 pivoted rows), BetMGM (10).
- **Markets:** Player Sets Won (60), Player Games Won (46), Player Break Points Won (12).
- **Scope:** 25 fixtures, 36 distinct players.
- Of the 118 pivoted rows, 106 have both sides and 12 have only one side.

Read this before modelling with it

This archive does not currently intersect the model. Two independent reasons:

1. **Tour.** The feature pipeline filters `player.tour = 'ATP'` and the ETL downloads `atp_matches_*.csv`. Every row here is WTA, so none of these players exist in the `players` table.
2. **Market.** The active model targets are `aces`, `double_faults`, and `service_games`. None of those appear here. The three markets present are Sets Won, Games Won, and Break Points Won — and the first two are the targets the model card lists as conceptually empty in the match store.

So a market-vs-model comparison cannot produce a paired row from this data, and continuing to collect under the current configuration will not change that. Loading it is still worth doing — it exercises the ingestion path and the archive cannot be bought retroactively — but expect a 0% player match rate until the tour question is resolved.

Columns — raw file

Identical to the capture archive, plus one addition.

Column	Notes
captured_at	Run timestamp, identical for every row in one night's capture. ISO 8601 UTC.
fixture_id,game_id	Provider fixture identifiers. Not Sackmann match ids.
league,tournament,start_date,status	Fixture context. <code>start_date</code> is match start.
home_player,away_player	Provider display names for the two players.
sportsbook	Book that offered the price.
market_id,market	e.g. <code>player_games_won / Player Games Won</code> .
player_id	Provider player id, a 16-character hex string. Not a Sackmann id.
selection	Provider player display name, e.g. <code>Anna Blinkova</code> . This is the name to resolve.
normalized_selection	Provider's own slug, e.g. <code>anna_blinkova</code> .
selection_line	<code>over</code> or <code>under</code> . This is the side of the line.
points	The line itself, e.g. <code>12.5</code> . Maps to <code>odds.line</code> .
price	American odds.
is_main	Whether this is the book's main line rather than an alternate.
grouping_key	Provider grouping, e.g. <code>anna_blinkova:12.5</code> .
odds_id	Provider odds identifier. Maps to <code>odds.provider_odds_id</code> , not to <code>odds.odds_id</code> , which is generated.
odds_timestamp	Unix epoch seconds, as returned.
odds_timestamp_utc	<i>Added here</i> . The same value as ISO 8601 UTC, for readability.

Columns — pivoted file

Same fixture and player context, then:

Column	Notes
provider_player_name, provider_player_id	Renamed from <code>selection / player_id</code> to make the provider origin obvious.
book,market,market_id,line, is_main	As above. <code>line</code> is the former <code>points</code> .
over_price,under_price	American odds for each side. Blank when the book only offered one side.
over_implied_prob, under_implied_prob	Computed locally from the American price: $100 / (p+100)$ for positive prices, $-p / (-p+100)$ for negative. Blank on one-sided rows.

Column	Notes
overround	The two implied probabilities summed. Above 1.0 by the book's margin.
over_novig_prob, under_novig_prob	Each implied probability divided by the overround, so the pair sums to 1.0. This is the book's opinion with its margin removed, and it is the number to compare a model probability against.
has_both_sides	True when both sides were present. De-vig columns are only populated when this is True.
over_odds_id, under_odds_id	Provider odds identifiers for each side.
odds_timestamp, odds_timestamp_utc	Price timestamp, epoch and ISO.

The derived probability columns are computed in this export rather than read from the provider, so they are worth spot-checking against one row before you trust them at scale.

Loading into Postgres

The pivoted file lines up with the odds table, but two things still need solving before a row will insert: `match_id` must come from resolving the fixture to a Sackmann match, and `player_id` must come from resolving `provider_player_name` to a Sackmann player. Migration 008 made `odds.player_id` nullable with a check that `provider_player_name` is present instead, so unresolved rows can be staged rather than dropped.

For a quick look without touching the schema, load into a staging table first:

```
CREATE TABLE IF NOT EXISTS odds_snapshot_stage (
  captured_at      TIMESTAMPTZ,
  league           TEXT,
  tournament       TEXT,
  start_date       TIMESTAMPTZ,
  fixture_id       TEXT,
  game_id          TEXT,
  home_player      TEXT,
  away_player      TEXT,
  provider_player_name TEXT,
  provider_player_id TEXT,
  normalized_selection TEXT,
  book             TEXT,
  market           TEXT,
  market_id        TEXT,
  line             NUMERIC(12,3),
  is_main          BOOLEAN,
  over_price        NUMERIC(12,3),
  under_price       NUMERIC(12,3),
  over_implied_prob NUMERIC(12,6),
  under_implied_prob NUMERIC(12,6),
  overround         NUMERIC(12,6),
  over_novig_prob   NUMERIC(12,6),
  under_novig_prob  NUMERIC(12,6),
  has_both_sides    BOOLEAN,
  over_odds_id      TEXT,
```

```
    under_odds_id      TEXT,
    grouping_key       TEXT,
    odds_timestamp     DOUBLE PRECISION,
    odds_timestamp_utc  TIMESTAMPTZ
);

\copy odds_snapshot_stage FROM 'tennis_prop_odds_pivoted_2026-09.csv' WITH (FORMAT csv);
```

Then check the join you actually care about:

```
-- How many provider names match a player already in the table?
SELECT
    s.league,
    count(*)                               AS rows_staged,
    count(p.player_id)                     AS rows_resolved,
    round(100.0 * count(p.player_id) / count(*), 1) AS pct_resolved
FROM odds_snapshot_stage AS s
LEFT JOIN players AS p
    ON lower(p.full_name) = lower(s.provider_player_name)
GROUP BY s.league;
```

Expect `pct_resolved` to come back at 0.0 for the reason in the section above. That is the correct result for this data, not a loading error.